APP DEV 2 PROJECT REPORT

1. Student Details

Name: Shubhankar Bajpai
Roll Number: 24F3005145
Email: 24f3005145@ds.study.iitm.ac.in
About Me: I am a student in the IIT Madras BS Degree Programme with a strong interest in full-stack web application development. I enjoy building scalable web applications using Flask and Vue.js while exploring concepts such as RESTful APIs, authentication, database management, caching, and asynchronous background processing.

---

2. Project Details

Project Title: TrekMate – Trekking Management Application V2

Problem Statement:

The objective of this project is to develop a web-based Trekking Management System that streamlines the management of trekking activities for administrators, trek staff, and trekkers. The system should provide secure role-based access, efficient trek management, online booking facilities, reporting capabilities, and background automation for notifications and scheduled tasks.

Approach:

The application follows a modular full-stack architecture using Flask for the backend and Vue.js for the frontend.

The backend exposes REST APIs for authentication, trek management, booking management, user management, reporting, and background processing. SQLite is used for persistent storage, while Redis is used for caching and Celery is used for asynchronous batch jobs.

The frontend is built using Vue 3 with Vue Router and Pinia to provide a responsive single-page application. Bootstrap 5 is used for styling and responsive layouts.

Role-Based Access Control (RBAC) is implemented to provide different functionality for Administrators, Trek Staff, and Trekkers.

---

3. AI/LLM Declaration

I used **ChatGPT (GPT-5)** to assist in discussing application architecture, debugging a few runtime errors, organizing code, generating boilerplate code and to get small suggestions on the UI of the app.
The extent of AI/LLM usage is around **15-20%**, limited to **code suggestions and documentation formatting**.

The overall implementation, testing, debugging, integration, feature selection, and final validation were performed manually.

---

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Backend REST API framework
SQLAlchemy	ORM for SQLite database
Flask-JWT-Extended	JWT Authentication
Flask-Migrate	Database migrations
Flask-Mail	Email notifications
Vue.js 3	Frontend SPA
Vue Router	Client-side routing
Pinia	State management
Axios	API communication
Bootstrap 5	UI and responsive design
SQLite	Database
Redis	API caching
Celery	Background job processing

---

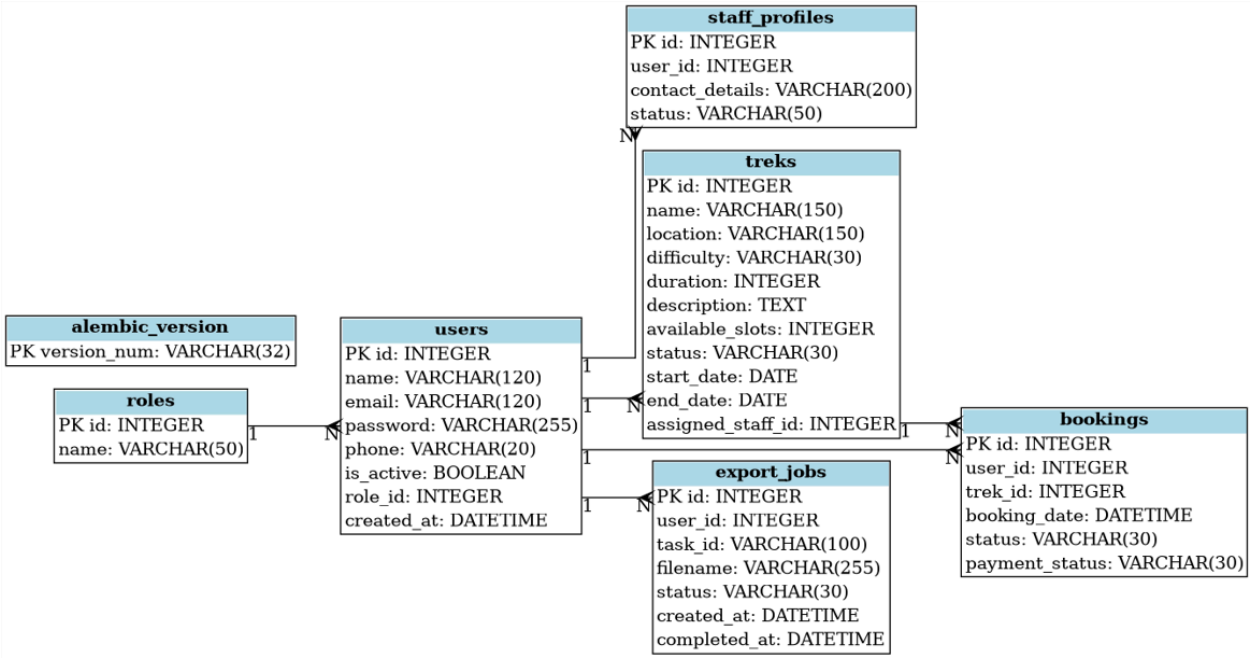
5. Database Schema / ER Diagram

Tables:

1. **Role:** *Stores system roles* (id, name)
2. **User:** *Stores all users* (id, name, email, password, phone, is_active, role_id, created_at)
3. **Trek:** *Stores trekking events* (id, name, location, difficulty, duration, description, available_slots, status, start_date, end_date, assigned_staff_id)
4. **Booking:** *Stores trek bookings* (id, user_id, trek_id, booking_date, status, payment_status)
5. **ExportJob:** *Stores background CSV export requests* (id, file_name, status, created_at, completed_at)
6. **StaffProfile:** *Stores all staff profiles* (id, user_id, contact_details, status)

Relationships:

- One-to-Many → `User → Booking`
- One-to-Many → `Trek → Bookings`
- One-to-Many → `User → Exportjob`
- One-to-Many → `Role → User` *(except for admin Role)*
- One-to-One → `User → StaffProfile`
- Many-to-Many → `User → Treks`



6. API Resource Endpoints

ENDPOINT	METHOD	DESCRIPTION
/api/auth/register	POST	Trekker registration
/api/auth/login	POST	User login
/api/admin/dashboard-summary	GET	Admin dashboard summary
/api/admin/treks	GET	List all treks
/api/admin/treks	POST	Create trek
/api/admin/staff	GET	List trek staff
/api/admin/staff	POST	Create trek staff
/api/admin/assign-staff/{id}	PUT	Assign staff to trek
/api/admin/users	GET	View all users
/api/admin/reports/statistics	GET	Reports and statistics
/api/staff/dashboard	GET	Staff dashboard
/api/staff/treks	GET	Assigned treks
/api/staff/trek/{id}	GET	Trek details
/api/staff/trek/{id}/participants	GET	Participant list
/api/user/treks	GET	Available treks
/api/user/bookings	GET	User bookings
/api/user/profile	GET/PUT	Profile management

YAML API Definition File:
Included separately in the submission ZIP as `api.yaml`.

---

7. Architecture and Features

Quick Architecture Overview:

Backend
│
├── Models
├── Routes
├── Services
├── Tasks
├── Utils
└── Extensions
Frontend
│
├── Components
├── Layouts
├── Router
├── Services

├── Stores
├── Views
├── Assests
└── Utils

Implemented Features:

Administrator, Trek Staff, Trekker Dashboards

Daily Trek Reminder Emails, Monthly Administrator Summary Report

CSV Export Generation, JWT Authentication, Role-Based Authorization

Redis Caching, Celery Background Jobs

Bootstrap Responsive UI, Toast Notifications

Automatic Logout for Deactivated Users

Secure password hashing and user session handling

---

8. Video Presentation

Drive Link:
https://drive.google.com/file/d/1DFfdBzmZjrzeTI6RvZskUCjm2TKbEd92/view?usp=sharing

---